TECHNICAL ARCHITECTURE WHITEPAPER

VoteX: Blockchain & Multi-Election Architecture

Comprehensive Guide to Smart Contracts, Cryptographic Commitments, and Scalable Multi-Tenancy

Platform: VoteX Digital Voting System | **Contract:** ElectionRegistry.sol (v0.8.20) | **Security:** Zero-Knowledge Anonymity | **Consensus:** EVM-Compatible

1. Executive Summary & The Problem Solved

Traditional digital voting systems rely entirely on centralized SQL databases. In a centralized architecture, an administrator or malicious actor with server credentials can execute:

```
UPDATE votes SET candidate_id = 'CAN-02' WHERE candidate_id = 'CAN-01';
```

This alteration leaves zero public forensic trace and cannot be detected by voters. **VoteX eliminates centralized trust** by decoupling ballot tallying from the server and anchoring all voting records to an immutable, decentralized blockchain smart contract.

> **Key Security Guarantee:** Once a ballot is mined into an Ethereum-compatible block, it cannot be modified, deleted, or backdated by anyone—not even the server administrator, university president, or platform creator.

2. The Core Cryptographic Paradox & Its Solution

Electronic elections face a classic mathematical contradiction:

- **Requirement A (Integrity):** The system must prove the voter is legally eligible and prevent them from voting more than once.
- **Requirement B (Anonymity):** The system must guarantee that nobody—under any circumstances—can discover *who* the voter voted for.

How VoteX Solves It: One-Way Cryptographic Commitments

Instead of sending the voter's name or student ID to the blockchain, VoteX generates a one-way mathematical commitment:

```
voterCommitment = SHA-256( Voter_ID + Server_Salt )
```

This generates a 64-character hexadecimal hash (e.g. `0x7f83b165...`). The blockchain contract stores this hash in an on-chain mapping:

```
election.hasVotedMapping[voterCommitment] = true;
```

Component	What is Stored on Blockchain?	Security / Privacy Impact
Voter Identity	`voterCommitment` (SHA-256 hash)	**100% Anonymous.** Mathematical one-way hash cannot be reversed to reveal the human voter.
Double Voting	`hasVotedMapping[commitment]`	**Blocked.** If the voter tries to submit a 2nd vote, the smart contract immediately reverts.
Candidate Vote	`election.voteCounts[candidateId] += 1`	**Decoupled.** Candidate counters increment independently without any link to the voter.
Audit Proof	`receipts[receiptCode] = VoteRecord`	**Verifiable.** The voter receives an on-chain receipt code to verify their vote was counted.

3. Smart Contract Deep Dive (ElectionRegistry.sol)

The core ledger logic is codified in `contracts/ElectionRegistry.sol` using Solidity `^0.8.20` and OpenZeppelin security primitives:

A. State Architecture

```
struct VoteRecord {
    bool exists;
    bytes32 proofHash;      // Certified biometric + timestamp proof
    uint256 timestamp;      // Exact block timestamp
}

struct Election {
    bool exists;
    string[] candidateIds;
    address admin;
    mapping(string => uint256) voteCounts;       // candidateId => tally
    mapping(bytes32 => bool) hasVotedMapping;    // voterCommitment => voted
    mapping(string => VoteRecord) receipts;      // receiptCode => proof
}

// Global mapping holding unlimited parallel elections on one contract
mapping(bytes32 => Election) private elections;
```

B. Atomic Vote Casting Function

```
function castVote(
    bytes32 electionId,
    string memory candidateId,
    bytes32 voterCommitment,
    bytes32 proofHash,
    string memory receiptId
) external {
    Election storage election = elections[electionId];
    require(election.exists, "Election does not exist");

    // Strict Anti-Double Voting Guard
    require(!election.hasVotedMapping[voterCommitment], "Voter has already voted");
    require(!election.receipts[receiptId].exists, "Receipt ID already used");

    // Candidate validation
    bool validCandidate = false;
    for (uint256 i = 0; i < election.candidateIds.length; i++) {
        if (keccak256(bytes(election.candidateIds[i])) == keccak256(bytes(candidateId))) {
            validCandidate = true;
            break;
        }
    }
    require(validCandidate, "Invalid candidate");

    // State Mutations
    election.hasVotedMapping[voterCommitment] = true;
    election.voteCounts[candidateId] += 1;

    uint256 currentTime = block.timestamp;
    election.receipts[receiptId] = VoteRecord({
        exists: true,
        proofHash: proofHash,
        timestamp: currentTime
    });

    emit VoteCast(electionId, voterCommitment, proofHash, receiptId, currentTime);
}
```

> **Event Emission:** The `emit VoteCast` event broadcasts the transaction to decentralized nodes across the network, enabling real-time zero-latency tally listeners and external auditing tools.

4. How VoteX Runs 50+ Simultaneous Elections

VoteX is engineered from the ground up for high-density multi-tenancy. A college campus, corporate enterprise, or municipality can conduct **50 distinct elections at the exact same minute** without interference.

1. **On-Chain Storage Partitioning:** In the smart contract, every election has its own unique `bytes32 electionId` key. Election #1 and Election #50 operate in completely isolated memory slots.
2. **Independent Voter Participation:** A student can vote in the Computer Science Election, the Sports Council Election, and the Campus Club Election. Because voting status is stored per room in `(election_id, voter_id, has_voted)`, casting a vote in Election #1 does NOT lock them out of Election #2.
3. **Delegated Admin RBAC:** 50 different department officers can manage their own candidate rosters, CSV voter rosters, and start/end time windows without access to other departments.

5. Complete Lifecycle of a Vote

The lifecycle of every ballot moves across five hardened layers:

Phase	Action Taken	Security Check
1. Authentication	Voter enters Room Number, Voter ID, and Phone Number.	Twilio Verify sends real SMS OTP. Server rejects invalid OTPs via carrier API.
2. Biometrics	Face vector (128D) or WebAuthn fingerprint sensor.	Mathematical Euclidean distance (≤ 0.54) & Anti-spoof liveness check.
3. Commitment	Generates `voterCommitment` & `proofHash`.	Decouples real voter name from ballot to protect voter privacy.
4. Blockchain Mining	Backend broadcasts `castVote()` to EVM ledger.	Validates candidate, sets `hasVoted = true`, records block timestamp.
5. Receipt Issuance	Voter receives `VTX-XXXXXXXX` receipt code & txHash.	Voter can independently verify their ballot inclusion on public ledger.

6. Dual-Mode Blockchain Engine (Production vs. Simulation)

VoteX incorporates a fault-tolerant hybrid architecture in `server/src/services/blockchainService.ts`:

- **Live Web3 Mode:** Connects to Ethereum Sepolia Testnet, Polygon PoS, or a local Hardhat node via `ethers.JsonRpcProvider`. Signs transactions using the admin private key and broadcasts them to the public mempool.
- **High-Fidelity Simulation Mode:** If the blockchain network experiences RPC latency or is offline during local demos, VoteX automatically switches to an in-memory EVM simulator. It produces cryptographic transaction hashes, increments block numbers, and strictly enforces duplicate-vote prevention without breaking the user experience.

7. Verification & Audit Transparency

How a Voter Verifies Their Vote

At any point during or after the election, a voter can visit the **Receipt Verification Portal** and enter their Receipt Code (e.g. `VTX-A8F291CB`). The system queries the blockchain function:

```
(bool exists, bytes32 proofHash, uint256 timestamp) = contract.verifyReceipt(electionId, receiptId);
```

The blockchain returns the cryptographic proof and the exact block confirmation timestamp, proving with mathematical certainty that their vote was counted in the final results.